

A Complete Teaching Guide to

The Economic Indicators Terminal

*From Zero to Bloomberg-Style Dashboard —
Every Line of Code Explained*

PREPARED FOR

Kristopher — Ghost Strategies LLC

TOPIC

Frontend architecture, data modeling, charting,
and live API integration — taught from first principles

TABLE OF CONTENTS

INTRODUCTION	3
PART I — FOUNDATIONS	
Chapter 01: What Actually Runs in a Browser	4
Chapter 02: The Three Languages	6
Chapter 03: How a Dashboard Is Structured	9
PART II — THE DATA LAYER	
Chapter 04: The DATA Object	11
Chapter 05: Economic Indicators Explained	13
Chapter 06: The Series ID System	15
PART III — THE VISUAL LAYER	
Chapter 07: CSS Variables and Design Tokens	17
PART IV — CHARTS & INTERACTION	
Chapter 10: Chart.js — How Charts Are Rendered	19
PART V — LIVE DATA INTEGRATION	
Chapter 13: HTTP, APIs, and the fetch() Function	22
Chapter 14: Connecting to FRED — Step by Step	25
PART VI — SETUP & DEPLOYMENT	
Chapter 17: Setting Up Your Machine	28
Chapter 18: Running Locally	32
Chapter 19: Deploying to GitHub Pages	34
PART VII — REFERENCE	
APPENDIX A: Series ID Dictionary	37
APPENDIX B: CSS Property Cheat Sheet	39
APPENDIX C: JavaScript Concepts Glossary	40
APPENDIX D: Troubleshooting Common Errors	41

INTRODUCTION

Welcome to the Terminal

Reading this guide is like taking a graduate seminar in frontend development, with the dashboard as your case study.

Who This Guide Is For

This guide assumes you know absolutely nothing about web development, JavaScript, APIs, or how dashboards are built. What it does assume is that you are comfortable reading financial content and thinking systematically. If you can read a 10-K, you can read this.

Everything is explained from first principles. When we mention a concept like "the DOM" or "a Promise," we define it at that moment. There are no prerequisites — you can start on page one with no context and reach page 70 with a working mental model of how every single line of the terminal was constructed.

What You Will Learn

By the end of this guide, you will be able to answer these questions with confidence:

- How does a browser turn text files into a visual dashboard?
- What is the difference between HTML structure, CSS styling, and JavaScript behavior?
- How do modern charting libraries like Chart.js actually draw pixels on screen?
- What is an API, and what does it mean to "call" one?
- Why do we need API keys, and how does authentication work in web apps?
- How does FRED compute "year-over-year percent change" for you, and why does that matter?
- What is Git, what is GitHub, and how does deploying to GitHub Pages work?
- Why does the dashboard use dark colors, JetBrains Mono, and orange accents specifically?

HOW TO READ

Each chapter builds on the last, but any chapter can stand alone as a reference. Code blocks are annotated line-by-line. Key terms appear in **bold** the first time they're used. When you see a callout box like this one, it's a practical tip or warning. The goal is that you finish this guide as a confident reader of frontend code.

CHAPTER 01

What Actually Runs in a Browser

The fundamental mental model: files travel, browsers interpret, pixels appear.

The Browser Is an Interpreter

When you open the file `index.html` in Chrome, Safari, or Firefox, something seemingly simple happens: a dashboard appears. Underneath that simplicity is a precise, well-defined process that has been standardized across every browser on Earth. Understanding this process is the foundation of everything that follows.

A web browser is, at its core, an **interpreter**. It takes three kinds of text files — HTML, CSS, and JavaScript — and translates them into the visual, interactive experience you see. The browser does not execute compiled programs the way Windows runs an `.exe` file. It reads plain text, parses it, and builds a running model of your page in memory.

The Three Phases of Loading

When the browser receives `index.html`, it performs three phases in sequence:

Phase 1: Parsing HTML into the DOM

The browser reads the HTML file top to bottom and constructs a tree-like structure in memory called the **Document Object Model**, or DOM for short. The DOM is the browser's internal representation of your page. Every tag you write — `<div>`, `<canvas>`, `<table>` — becomes a node in this tree.

Think of the DOM as an org chart for your webpage. The `<html>` tag is the CEO. Directly beneath it are `<head>` and `<body>`. Under `<body>` sit all your visible elements. JavaScript can later reach into this tree and manipulate any node.

Phase 2: Applying CSS Styles

Next, the browser reads all CSS — whether it's in a separate file, inside a `<style>` tag, or attached to an element's style attribute. For each element in the DOM, the browser figures out what color, font, size, position, and spacing it should have. This phase produces the **render tree**, which is the DOM combined with computed styles for every node.

Crucially, browsers resolve style conflicts through **specificity rules**. If two CSS rules both target the same element, the more specific one wins. An ID selector beats a class selector, which beats an element selector. This is why our terminal uses CSS variables at the `:root` level — they're inherited by everything but can be overridden locally when needed.

Phase 3: Executing JavaScript

Finally, the browser runs any JavaScript it encounters. By default, JavaScript runs synchronously — the browser stops parsing HTML, executes the script, then resumes. This is why our terminal puts most JavaScript *after* the HTML body: we want the DOM fully built before we try to query it.

JavaScript then takes over. It can modify the DOM (adding, removing, or changing elements), respond to clicks and keyboard events, fetch data from APIs, and animate elements. Everything you interact with after the page loads is JavaScript doing its job.

**MENTAL
MODEL**

HTML is the skeleton. CSS is the skin and clothing. JavaScript is the muscles and brain. A page with no CSS looks like a 1994 academic webpage. A page with no JavaScript is a poster — beautiful but inert. The terminal uses all three, and each has a specific job.

CHAPTER 02

The Three Languages

HTML, CSS, and JavaScript — what each one is and what it is not.

HTML — The Structure

HTML stands for HyperText Markup Language. It is not a programming language — it cannot do math, store variables, or make decisions. It is a **markup language**, which means its entire job is to tell the browser "here is a piece of content, and here is what kind of content it is."

An HTML document is made of **tags**. Every tag comes in a pair: an opening tag and a closing tag. Whatever sits between them is the content of that tag. Here is a minimal example from the terminal:

```
<div class="panel">
  <div class="panel-header">
    <div class="panel-title">CPI & PPI Trend</div>
  </div>
  <div class="chart-container">
    <canvas id="chartCpiPpi"></canvas>
  </div>
</div>
```

This snippet creates a panel (a box) with a title and a canvas element inside. The `<canvas>` tag is special — it's a blank rectangular area that JavaScript can later draw graphics onto. Chart.js uses this canvas to render our CPI and PPI line charts pixel by pixel.

Notice the class and id attributes. A **class** is a label you can apply to many elements — every panel in the terminal has `class="panel"`, so they all get the same styling. An **id** must be unique per page — we use `id="chartCpiPpi"` because JavaScript needs a unique way to find this one specific canvas among many.

CSS — The Appearance

CSS stands for Cascading Style Sheets. Its job is to answer one question for each HTML element: "What should this look like?" CSS rules pair a **selector** (which elements to target) with **declarations** (what styles to apply).

```
.panel {
  background: var(--bg-card);
  padding: 16px;
  border: 1px solid var(--border);
}

.panel-title {
  font-family: 'JetBrains Mono', monospace;
  font-size: 11px;
  color: var(--accent-orange);
  text-transform: uppercase;
  letter-spacing: 1.5px;
}
```

Let's dissect this. The `.panel` rule matches every element with `class="panel"` — the leading dot means "class." It sets three properties: a dark background color, 16 pixels of internal padding, and a thin border. The second rule styles panel titles with a monospace font, small uppercase letters, and the signature orange color.

The `var(--bg-card)` syntax is a **CSS custom property** or "CSS variable." We define these once at the top of the file and reference them everywhere. This means changing the dashboard from dark to light theme would only require editing a handful of variable definitions, not hundreds of individual color declarations. We'll explore this in depth in Chapter 7.

The Cascade

The "C" in CSS stands for **Cascading**. When multiple rules could apply to the same element, the browser follows a precise algorithm to decide which wins:

- **Specificity:** `#id` beats `.class` beats tag.
- **Source order:** if specificity is equal, the later rule wins.
- **Importance:** `!important` overrides normal rules (use sparingly).
- **Inheritance:** some properties like color and font-family are inherited from parent elements.

JavaScript — The Behavior

JavaScript (unrelated to the Java programming language despite the name) is a full programming language that runs inside browsers. It can do arithmetic, store data in variables, define functions, make decisions with if statements, loop over lists, and — most importantly for us — manipulate the DOM and call external APIs.

Variables and Data

```
// Three ways to declare a variable:
const cpi = 2.8;           // cannot be reassigned
let inflation = 'high';    // can be reassigned
var legacy = true;         // old syntax, avoid in new code

// Objects — key-value pairs
const dataPoint = {
  date: '2026-01-01',
  value: 3.1,
  indicator: 'CPI'
};

// Arrays — ordered lists
const quarters = ['Q1', 'Q2', 'Q3', 'Q4'];
```

In the terminal, we use `const` for almost everything. This communicates to anyone reading our code that these values don't change after they're set. `let` is reserved for values that genuinely need to change, like counters in a loop.

Functions

A **function** is a reusable block of code. Define it once, call it many times. JavaScript has two common syntaxes:

```
// Traditional function declaration
function formatPercent(value) {
  return value.toFixed(1) + '%';
}

// Arrow function (modern, more concise)
const formatPercent = (value) => value.toFixed(1) + '%';
```

```
// Calling the function
const displayValue = formatPercent(2.847); // returns '2.8%'
```

The terminal defines dozens of functions — `buildSummaryCards()`, `renderView()`, `fetchFRED()`, and so on. Each has a single clear purpose. This is the principle of **separation of concerns**: each piece of code does one thing well, making the whole system easier to reason about and modify.

Asynchronous Operations

The most conceptually difficult part of JavaScript is **asynchrony**. When we call an API to fetch data from FRED, that request travels over the internet and may take hundreds of milliseconds to return. JavaScript doesn't freeze the browser while waiting — instead, it registers a callback and continues running. When the data arrives, the callback fires.

Modern JavaScript handles this with **async/await** syntax, which makes asynchronous code look almost synchronous:

```
async function loadData() {
  const response = await fetch('https://api.example.com/data');
  const json = await response.json();
  return json;
}
```

The `await` keyword tells JavaScript: pause here until the promise resolves, then continue. Without `async/await`, we'd be nesting callbacks inside callbacks — a pattern so ugly it earned the nickname "callback hell" in the 2010s.

CHAPTER 03

How a Dashboard Is Structured

From blank page to Bloomberg clone — the anatomy of the terminal.

The Skeleton

Every HTML file begins with a standard boilerplate. Here's the terminal's skeleton, stripped of content:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>GHOST STRATEGIES | Economic Indicators Terminal</title>
  <script src="https://cdnjs.cloudflare.com/.../chart.umd.min.js"></script>
  <link href="https://fonts.googleapis.com/css2?family=JetBrains+Mono..." rel="stylesheet">
  <style>
    /* All CSS rules live here */
  </style>
</head>
<body>
  <!-- All visible content goes here -->
  <script>
    /* All JavaScript logic lives here */
  </script>
</body>
</html>
```

Each line has a specific purpose. `<!DOCTYPE html>` tells the browser to use modern HTML5 rules. `<meta charset="UTF-8">` ensures special characters (the orange ▲ arrows in our data tables, for instance) render correctly. `<meta viewport>` makes the page responsive on mobile devices.

Loading External Dependencies

Our terminal depends on two external libraries loaded from Content Delivery Networks (CDNs):

Chart.js — The Charting Library

```
<script src="https://cdnjs.cloudflare.com/.../chart.umd.min.js">
```

This line fetches Chart.js, a popular open-source charting library, from Cloudflare's CDN. Once loaded, a global variable called `Chart` becomes available to all our JavaScript code. The library itself is around 200 KB and takes less than a second to download on a typical connection.

Using a CDN instead of hosting Chart.js ourselves has two benefits. First, Cloudflare's servers are faster and more geographically distributed than any single server we could set up. Second, if the user has visited another site that also uses Chart.js from the same CDN, the library may already be in their browser cache, loading instantly.

Google Fonts — Typography

```
<link href="https://fonts.googleapis.com/css2?family=JetBrains+Mono...">
```

This imports two typefaces: **JetBrains Mono** (a monospace font designed for code, used for our headers and numbers) and **IBM Plex Sans** (a humanist sans-serif, used for body text). Google Fonts serves these files with proper caching headers so they load quickly across visits.

The Six Major Sections

Inside `<body>`, the terminal is organized into six major sections:

- **Top Bar** — Ghost Strategies logo, live status indicator, data source labels, and a real-time clock.
- **Ticker Bar** — Horizontally scrolling strip of headline numbers, animated with a CSS keyframe.
- **Navigation Tabs** — Six tabs (Overview, CPI Detail, PPI Detail, Employment, Projections, Correlation Matrix) that switch the main view.
- **Summary Cards** — A grid of six large tiles showing the headline number for each major indicator.
- **Main Dashboard** — The dynamic area that rerenders based on which tab is active. Contains charts, tables, and matrices.
- **Footer** — Terminal metadata: product name, data sources, auto-refresh countdown.

This structure mirrors how Bloomberg, Reuters, and most professional financial dashboards organize information. The top bar and ticker are always visible (anchored with `position: sticky`). Navigation sits below. The main content changes based on user input. The footer provides persistent context.

SINGLE FILE

The entire terminal is one HTML file. No separate CSS or JavaScript files, no build step, no bundler. This is a deliberate choice: single-file dashboards are trivial to share, archive, version-control, and email. The downside is slightly less separation of concerns, but at this scale (around 1,400 lines) it's manageable and the benefits are significant.

CHAPTER 04

The DATA Object

How the terminal models economic data in JavaScript.

Why We Need a Data Model

Before we can display CPI on a chart, we need to represent it somehow in the program. This is called **data modeling**, and it's one of the most important decisions in any software project. Get the data model right, and everything else becomes easy. Get it wrong, and you'll fight your own code forever.

In the terminal, all economic data lives inside a single JavaScript object called DATA. This object is the single source of truth. Every chart, every table, every ticker value reads from DATA. When we eventually wire up live FRED calls, they don't replace the dashboard — they just update DATA, and the dashboard re-renders automatically.

The Structure of a Single Indicator

Here's how CPI is modeled:

```
const DATA = {
  cpi: {
    label: 'CPI-U All Items',
    unit: '%',
    annual: {
      2020: 1.2, 2021: 4.7, 2022: 8.0,
      2023: 4.1, 2024: 2.9, 2025: 2.6
    },
    quarterly: {
      '2023-Q1': 6.0, '2023-Q2': 4.0,
      '2023-Q3': 3.6, '2023-Q4': 3.2,
      '2024-Q1': 3.2, '2024-Q2': 3.3,
      /* ...many more quarters... */
    },
    projected: {
      '2026-Q2': 2.3,
      '2026-Q3': 2.2,
      '2026-Q4': 2.1
    },
    latest: 2.8,
    prevMonth: 2.9,
    mom: 0.2,
    target: 2.0,
    releaseDate: '2026-03-12'
  },
  // ...more indicators...
};
```

Dissecting the Pattern

Every indicator in DATA follows the same shape, which makes the code that consumes them simple and uniform. Let's walk through each field:

- **label** — Human-readable name shown in charts and tables.
- **unit** — What unit the values are in (% for rates, K for thousands of jobs).
- **annual** — A dictionary keyed by year, containing annual averages.
- **quarterly** — A dictionary keyed by year-quarter strings like '2024-Q1'.
- **projected** — Future quarters, stored separately so charts can render them with dashed lines.
- **latest** — The most recent reading, used by summary cards and tickers.
- **prevMonth** — Previous reading, used to compute directional changes.
- **mom** — Month-over-month percent change.
- **target** — Policy target (for CPI, this is the Fed's 2% goal).
- **releaseDate** — When the data was officially released.

This pattern is called a **normalized schema**. Every indicator — CPI, Core CPI, PPI, NFP, unemployment, wages, JOLTS — has the same fields. When we write a function to render a summary card, we can call it with any indicator:

```
function renderCard(indicator) {  
  return `  
    <div class='card'>  
      <div class='label'>${indicator.label}</div>  
      <div class='value'>${indicator.latest}${indicator.unit}</div>  
      <div class='change'>${indicator.latest - indicator.prevMonth}</div>  
    </div>  
  `;  
}  
  
renderCard(DATA.cpi);      // works  
renderCard(DATA.ppi);      // works  
renderCard(DATA.unrate);   // works
```

Template Literals — The Backtick Syntax

Notice the backticks (`) surrounding the HTML string above. These are **template literals**, a modern JavaScript feature. Unlike regular strings with single or double quotes, template literals can span multiple lines and can embed expressions with `\${...}` syntax.

In the example, `\${indicator.label}` is replaced at runtime with the actual label value. This is how the terminal dynamically generates HTML — not by manipulating DOM nodes one at a time, but by building strings and inserting them wholesale.

Why Objects Instead of Arrays?

A natural question: why store quarterly data as `{ '2024-Q1': 3.2, '2024-Q2': 3.3 }` instead of `[ { quarter: '2024-Q1', value: 3.2 }, ... ]`? Both representations carry the same information. The object-based approach was chosen for three reasons:

- **Direct lookups**: `DATA.cpi.quarterly['2024-Q3']` instantly gives us 3.6. With an array, we'd need to search.
- **Uniqueness enforcement**: you literally cannot have two "2024-Q1" entries — the object structure prevents it.
- **Easy serialization**: this format converts directly to JSON for storage or API responses without extra work.

When we need to iterate over quarters (for building a table or a chart), JavaScript's `Object.keys()` gives us the list of quarter strings in insertion order, and `Object.entries()` gives us both keys and values. So we get the best of both worlds.

CHAPTER 05

Economic Indicators Explained

What each number in the terminal actually measures.

CPI — The Consumer Price Index

The **Consumer Price Index**, published monthly by the Bureau of Labor Statistics, measures the average change in prices paid by urban consumers for a fixed basket of goods and services. It is the most widely-watched inflation gauge in the United States and drives Social Security adjustments, Treasury Inflation-Protected Securities (TIPS), and many collective bargaining agreements.

The terminal tracks two versions: **Headline CPI** (all items) and **Core CPI** (excluding food and energy). Core CPI exists because food and energy prices are notoriously volatile — a cold snap in Brazil can spike coffee prices; oil shocks whipsaw energy costs. Stripping these out gives a cleaner view of underlying inflation trends, which is why the Federal Reserve's favorite inflation measure (Core PCE) similarly excludes them.

How CPI Is Computed

BLS sends data collectors to thousands of retail outlets each month, recording prices on roughly 80,000 items across 200+ categories. These prices are weighted according to the Consumer Expenditure Survey — which is why shelter, at 36.5% of the basket, dominates the index. Each category's price change is multiplied by its weight, and the sum gives headline CPI.

The shelter weight is why economists pay obsessive attention to the rent components — Rent of Primary Residence (about 7.5% of CPI) and Owners' Equivalent Rent (about 27%). These move slowly because landlords typically reset rents annually, creating a well-known lag between actual market rents (measured by Zillow, Apartment List, and similar indexes) and what shows up in CPI. This lag is central to the current disinflation story: market rents peaked in mid-2022, and CPI shelter is still catching up.

PPI — The Producer Price Index

The **Producer Price Index** measures the average change in selling prices received by domestic producers. Where CPI asks "what do households pay?", PPI asks "what do businesses charge?" The two are related but distinct — PPI captures wholesale, factory-gate, and intermediate prices before retail markup.

PPI often leads CPI by one to three months. When input costs rise at the producer level, those increases eventually pass through to consumers — unless competition or demand destruction absorbs them. The terminal's **PPI-to-CPI Pipeline** chart in the PPI Detail view visualizes this relationship directly, showing the spread between PPI and CPI quarter by quarter.

NFP — Nonfarm Payrolls

Nonfarm Payrolls is the headline number from the BLS **Employment Situation** report, released on the first Friday of each month at 8:30 AM Eastern. It measures the net change in jobs across all sectors except farming, private

households, and the military. It is arguably the single most market-moving economic release in the world — treasury yields can move 10+ basis points in the first minute after release.

The terminal displays NFP in three ways. Summary cards show the most recent month's change. The quarterly chart sums three months of changes into a quarterly total. The sector breakdown table (in the Employment view) shows which industries added or shed jobs in the most recent month.

Unemployment Rates — U-3 and U-6

The headline **unemployment rate** you see on news tickers is **U-3**: the percentage of the civilian labor force that is without a job but actively searching. **U-6** is a broader measure — it includes U-3 plus "marginally attached" workers (those who have stopped looking but want a job) and workers involuntarily working part-time who'd prefer full-time employment.

The gap between U-3 and U-6 is a signal of labor market slack. When U-6 is much higher than U-3, there's more hidden unemployment — people who technically aren't counted as unemployed but are underemployed or discouraged. The terminal tracks both in the Employment view.

JOLTS — Job Openings and Labor Turnover

The **JOLTS** survey measures job openings, hires, and separations. Economists watch it for the **openings-to-unemployed ratio** — when this is above 1.0, there are more open jobs than people looking, indicating a tight labor market. During 2022, this ratio hit an unprecedented 2.0, which helped justify the Fed's aggressive rate hikes.

Average Hourly Earnings

Average Hourly Earnings (AHE) measures wage growth. The terminal tracks year-over-year percent change. When AHE exceeds CPI, workers are experiencing real wage growth — their paychecks outpace inflation. When CPI exceeds AHE, real wages are falling even if nominal pay is rising. The **Real Wage Growth** chart in the Employment view plots this spread directly.

FOR YOUR WORK

At Toast as a Senior Capital Underwriter, you likely model CPI as an input to consumer spending forecasts, and NFP as a proxy for hiring in hospitality (which is a large Toast vertical). The sector-level employment data in the terminal is particularly relevant — "Leisure & Hospitality" job growth is a leading indicator for Toast's own merchant acquisitions.

CHAPTER 06

The Series ID System

How government agencies identify data series.

Why Series IDs Exist

Every economic data series published by BLS or FRED has a unique alphanumeric code called a **series ID**. These IDs look intimidating at first — CUSR0000SA0L1E is hardly memorable — but they follow a logic. Understanding that logic unlocks direct access to thousands of series without ever leaving the BLS documentation.

Decoding a BLS CPI Series ID

Take CUSR0000SA0L1E, which is Core CPI. Each segment has meaning:

```
CU  SR  0000  SA0L1E
|   |   |     |
|   |   |     +-- Item code: SA0L1E = All items less food and energy
|   |   +----- Area code: 0000 = U.S. city average
|   +----- Seasonal adjustment: SR = Seasonally adjusted
+----- Survey: CU = CPI for Urban Consumers
```

With this decoder, you can construct any CPI series ID without looking it up. Want CPI for the New York metropolitan area? Change the area code. Want the unadjusted version? Change SR to UR. This systematic naming is standard across BLS surveys.

FRED Series IDs

FRED uses simpler, more memorable IDs that don't follow a strict encoding scheme. Here are the most important ones for the terminal:

Indicator	FRED ID	Description
CPI All Items	CPIAUCSL	Seasonally adjusted, urban consumers
Core CPI	CPILFESL	Less food and energy
PPI Final Demand	PPIFIS	Headline PPI
Core PPI	PPIFES	Less food, energy, trade
Nonfarm Payrolls	PAYEMS	Total NFP, thousands
Unemployment Rate	UNRATE	U-3 civilian rate
U-6 Rate	UGRATE	Broader unemployment
LFPR	CIVPART	Labor force participation

Avg Hourly Earnings	CES0500000003	Private nonfarm
JOLTS Openings	JTSJOL	Total job openings
Fed Funds Rate	FEDFUNDS	Effective federal funds rate
10Y Treasury	DGS10	10-year constant maturity

FRED aggregates data from hundreds of sources, not just BLS. FEDFUNDS comes from the Federal Reserve Board. DGS10 comes from Treasury. GDPC1 (real GDP) comes from BEA. This is what makes FRED so powerful — one API, one authentication, access to virtually every important U.S. economic series.

Discovering New Series

When you need a series the terminal doesn't include — say, the Case-Shiller Home Price Index — the workflow is always the same:

- Go to fred.stlouisfed.org and search for the series name.
- Open the series page; the series ID appears in parentheses after the title and in the URL.
- Add the ID to your FRED fetch calls with the appropriate units (lin, pc1, pch).
- Update the DATA object schema to include the new series in the format the dashboard expects.

This is how the terminal can grow from tracking 15 indicators today to 50 or 500 in the future with only incremental work.

CHAPTER 07

CSS Variables and Design Tokens

Why your dashboard uses --bg-primary instead of #0a0e14.

The Problem They Solve

Imagine you've built a dashboard with 800 CSS rules, and every rule that uses your brand color has `color: #ff9500` hard-coded. Now marketing decides to tweak the brand to `#ff8800`. You now face a tedious find-and-replace that must be applied consistently, and you have to pray you didn't miss a spot or change a similar orange that's meant to be different.

CSS custom properties — commonly called CSS variables — solve this by letting you define a value once and reference it everywhere. Change the definition, and every reference updates automatically. This is one of the most important modern CSS features.

Definition Syntax

```
:root {
  --bg-primary: #0a0e14;
  --bg-card: #111820;
  --accent-orange: #ff9500;
  --accent-green: #00c853;
  --text-primary: #d4d8de;
}
```

The `:root` selector matches the HTML document's root element (same as `html`, but with slightly higher specificity). Defining variables here makes them available everywhere in your CSS.

The double-hyphen prefix (`--`) is mandatory and distinguishes custom properties from built-in CSS properties.

Usage Syntax

```
.summary-card {
  background: var(--bg-card);
  color: var(--text-primary);
  border: 1px solid var(--border);
}

.summary-card .value {
  color: var(--accent-orange);
}
```

The `var()` function retrieves a custom property's value. You can also provide a fallback: `var(--unused, #ff0000)` uses red if `--unused` isn't defined. This is useful for defensive coding when you're not sure if a variable will exist in every context.

The Terminal's Design Token System

The terminal defines roughly 15 CSS variables. Think of them as the dashboard's design vocabulary:

Variable	Value	Semantic Meaning
--bg-primary	#0a0e14	Page background
--bg-secondary	#0f1419	Top bar, nav bar
--bg-card	#111820	Panel and card backgrounds
--bg-tertiary	#151b23	Table headers, nested elements
--border	#1e2a36	Standard borders
--border-accent	#2a3a4a	Emphasized borders
--text-primary	#d4d8de	Main text
--text-secondary	#7a8899	Secondary text, labels
--text-muted	#4a5666	Metadata, disclaimers
--accent-orange	#ff9500	Brand primary, highlights
--accent-blue	#0084ff	CPI core, secondary data
--accent-green	#00c853	Positive values, gains
--accent-red	#ff3b30	Negative values, warnings
--accent-cyan	#00e5ff	Tertiary data, time display
--accent-purple	#bb86fc	Projected/forecast data

The Semantic Naming Pattern

Notice that variables are named by role, not by appearance. We say `--bg-card`, not `--dark-blue-grey`. This is intentional. Semantic names survive design changes: if we later redesign the terminal with a lighter "card" color, we only change the value, not the name. If we had named it by appearance, we'd have to rename every reference or end up with a variable called `--dark-blue-grey` that's actually pink.

Enabling Theming

Because the terminal's entire color palette is driven by these 15 variables, adding a light theme is trivial:

```
/* Default: dark theme */
:root {
  --bg-primary: #0a0e14;
  --text-primary: #d4d8de;
  /* ... */
}

/* Light theme: override when body has class 'light' */
body.light {
  --bg-primary: #ffffff;
  --text-primary: #1a1a1a;
  /* ... */
}
```

Adding a theme toggle button that applies `body.light` would instantly switch the entire dashboard's color scheme. This is the same technique used by GitHub, Vercel, Linear, and most modern web apps.

CHAPTER 10

Chart.js — How Charts Are Rendered

From JavaScript objects to visual graphics.

Canvas — The Drawing Surface

The HTML `<canvas>` element is special. Unlike `<div>` or `<p>`, which contain structured content, canvas is a blank pixel buffer. JavaScript can draw anything on it — lines, curves, text, gradients — using an API similar to the 2D drawing APIs in macOS Quartz or Windows GDI.

You could draw charts by hand using the raw Canvas API, but you'd spend weeks implementing things that every chart needs: axes, tick marks, gridlines, tooltips, legends, animations, responsive sizing. Chart.js is a library that does all of this for you. You describe what you want in a configuration object, and Chart.js handles the drawing.

The Chart Configuration Object

Every Chart.js chart is created by passing two arguments: a canvas element, and a configuration object. Here is a simplified version of the CPI/PPI chart:

```
const ctx = document.getElementById('chartCpiPpi');

new Chart(ctx, {
  type: 'line',
  data: {
    labels: ['2024-Q1', '2024-Q2', '2024-Q3', '2024-Q4'],
    datasets: [
      {
        label: 'CPI',
        data: [3.2, 3.3, 2.6, 2.7],
        borderColor: '#ff9500',
        tension: 0.3
      },
      {
        label: 'PPI',
        data: [1.6, 2.4, 2.0, 3.0],
        borderColor: '#ffd600',
        tension: 0.3
      }
    ]
  },
  options: {
    responsive: true,
    plugins: { legend: { position: 'bottom' } }
  }
});
```

Anatomy of the Configuration

type: Tells Chart.js what kind of chart to draw. Options include line, bar, pie, doughnut, radar, scatter, and more.

data.labels: An array of strings that label the x-axis. The length of this array determines how many data points each dataset should have.

data.datasets: An array of data series to plot. Each dataset has its own data array, color, and styling options. Multiple datasets on the same chart share the same x-axis labels.

options.responsive: When true, the chart automatically resizes when its container does. Essential for making the dashboard work on different screen sizes.

options.plugins: Configuration for Chart.js plugins like the legend, tooltip, and title. Setting `legend.position = 'bottom'` moves the legend below the chart — common in financial dashboards where vertical space is precious.

The Dual-Axis Technique

The Employment view contains a particularly elegant chart: bars for NFP job change (measured in thousands) and a line for the unemployment rate (measured in percent). These two series have wildly different scales — NFP values like 600 and unemployment values like 4.1 cannot share a single y-axis without one completely dominating the visual.

```
options: {
  scales: {
    y: {
      position: 'left',
      title: { text: 'NFP (thousands)' }
    },
    y1: {
      position: 'right',
      grid: { display: false },
      ticks: { callback: v => v + '%' }
    }
  }
}
```

We define two y-axes — `y` on the left for NFP, `y1` on the right for unemployment. Each dataset specifies which axis it uses via `yAxisID`. The `grid: { display: false }` on the right axis prevents two overlapping gridlines, which would look cluttered.

Rendering Projected Data

A design challenge specific to economic dashboards: how do you show historical data and projected data on the same line without making them look like the same thing? The terminal's solution is to use two datasets per indicator — one for history, one for projections — where the projection dataset uses `borderDash: [5, 5]` to create a dashed line:

```
{
  label: 'CPI (Historical)',
  data: [3.2, 3.3, 2.6, 2.7, 2.8, 2.7, null, null, null],
  borderColor: '#ff9500',
  borderWidth: 2
},
{
  label: 'CPI (Projected)',
  data: [null, null, null, null, null, 2.7, 2.3, 2.2, 2.1],
  borderColor: '#ff9500',
  borderDash: [5, 5],
  borderWidth: 1.5
}
```

The null values tell Chart.js to skip those points. The projection dataset starts at the last historical value (to connect the lines visually) and continues with forecast values. Both datasets share the same color but the dashes clearly signal which portion is observed versus predicted. This technique is used by the Federal Reserve's own Summary of Economic Projections charts.

**WHY CHART
.JS
NOT D3**

D3.js is more powerful but requires hundreds of lines of code for a basic chart. Chart.js trades flexibility for convenience — perfect for dashboards where you need many standard charts quickly. The terminal renders 12+ charts in under 300 lines of Chart.js config.

CHAPTER 13

HTTP, APIs, and the fetch() Function

How your browser talks to a government server.

What HTTP Actually Is

Every time your browser requests a page, image, or piece of data from the internet, it speaks a protocol called **HTTP** (HyperText Transfer Protocol), or its encrypted cousin **HTTPS**. Despite the name, HTTP is not limited to hypertext — it carries images, video, JSON, PDFs, and anything else. It is the universal language of the web.

An HTTP request has four components: a **method** (verb), a **URL**, **headers** (metadata), and optionally a **body** (payload). A response has a **status code**, headers, and usually a body. Here's what the browser sends when it fetches CPI data:

```
GET /fred/series/observations?series_id=CPIAUCSL&api_key=YOUR_KEY HTTP/1.1
Host: api.stlouisfed.org
Accept: application/json
User-Agent: Mozilla/5.0 ...
```

The server responds with JSON data, looking something like:

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "observation_start": "2020-01-01",
  "observations": [
    {"date": "2020-01-01", "value": "2.5"},
    {"date": "2020-02-01", "value": "2.3"},
    ...
  ]
}
```

Status Codes

The status code in the response tells your code whether the request succeeded. The most important ones for API work are:

- **200 OK** — Success, response body contains data.
- **400 Bad Request** — Your request was malformed (missing parameter, invalid format).
- **401 Unauthorized** — API key missing or invalid.
- **403 Forbidden** — API key valid but doesn't have permission for this resource.
- **404 Not Found** — The series ID or endpoint doesn't exist.
- **429 Too Many Requests** — You hit the rate limit; slow down.
- **500 Internal Server Error** — Something broke on their end, not yours.

APIs — Application Programming Interfaces

An **API** is simply a set of HTTP endpoints that return structured data. Where a website serves HTML pages meant for human eyes, an API serves JSON (or XML) meant for programs. FRED's API is a collection of endpoints, each serving a different kind of query — one for series observations, another for series metadata, another for searching.

The critical insight: APIs are just HTTP. There is no separate "API protocol." If you can open a URL in your browser, you can call an API — they're the same thing under the hood. You can literally paste a FRED API URL with your key into Chrome and see the JSON response directly.

The `fetch()` Function

`fetch()` is JavaScript's built-in function for making HTTP requests. It returns a **Promise**, which is JavaScript's way of representing an operation that will complete in the future. Here's the pattern the terminal uses:

```
async function fetchFRED(seriesId) {
  const url = `https://api.stlouisfed.org/fred/series/observations`
    + `?series_id=${seriesId}`
    + `&api_key=${FRED_API_KEY}`
    + `&file_type=json`
    + `&units=pc1`
    + `&frequency=q`;

  const response = await fetch(url);
  const json = await response.json();

  return json.observations.map(o => ({
    date: o.date,
    value: parseFloat(o.value)
  }));
}
```

Walking Through Line by Line

Line 1: `async function` marks this function as asynchronous. It enables the use of `await` inside and makes the function return a Promise.

Lines 2-7: We build the URL by concatenating strings. The `${...}` syntax inside backtick-quoted strings is template literal interpolation — it substitutes the variable value into the string at runtime.

Line 9: `await fetch(url)` sends the HTTP request and pauses execution until the response arrives. The result is a Response object — not the actual data yet, but a wrapper around the response with methods to read the body.

Line 10: `await response.json()` reads the response body as JSON and parses it into a JavaScript object. This is also `async` because reading the body is itself a network operation for large responses.

Lines 12-15: We transform the API's response format into the format our dashboard wants. `.map()` applies a function to every element of an array. We pluck the date field as-is and convert the string value to a number with `parseFloat`. This transformation step is crucial — APIs return data in their preferred format, and we reshape it into ours.

Why JSON?

JSON (JavaScript Object Notation) is the standard data format for modern APIs. It's human-readable, maps cleanly onto JavaScript objects, and is supported natively by every programming language. The `response.json()` call parses the incoming JSON string into a JavaScript object you can navigate with dot notation.

Older APIs (SOAP, XML-RPC) used XML, which is more verbose and harder to work with. A handful of APIs still offer XML, but virtually every new API in the last decade is JSON-first.

**TRY IT
YOURSELF**

Take a FRED URL with your API key and paste it directly into a browser tab. You'll see raw JSON. This is exactly what `fetch()` sees. Understanding that APIs are just URLs demystifies everything — there's no hidden protocol, no special magic, just HTTP requests and JSON responses.

CHAPTER 14

Connecting to FRED — Step by Step

From sign-up to live data flowing into the terminal.

Step 1: Register for an API Key

Navigate to https://fred.stlouisfed.org/docs/api/api_key.html. If you don't have a FRED account, you'll be prompted to create one — email plus password. Once logged in, click "Request API Key." Enter a short description of your application (Ghost Strategies Economic Terminal works) and submit.

FRED issues the key immediately. It's a 32-character lowercase alphanumeric string — something like a1b2c3d4e5f6789012345678abcdef01. Copy it and save it somewhere secure. You'll need it in step 3.

Step 2: Understand the FRED URL Structure

Every FRED API call follows this pattern:

```
https://api.stlouisfed.org/fred/series/observations
?series_id=CPIAUCSL
&api_key=YOUR_KEY
&file_type=json
&observation_start=2020-01-01
&units=pc1
&frequency=q
&sort_order=asc
```

Each parameter after the ? is called a **query parameter**. They're separated by & and come in name=value pairs. Let's decode each one:

- **series_id**: Which series to fetch. CPIAUCSL is CPI.
- **api_key**: Your authentication — tells FRED who you are.
- **file_type**: What format to return. json is the modern choice.
- **observation_start**: Earliest date to include (YYYY-MM-DD format).
- **units**: Transformation. pc1 = percent change from year ago. pch = percent change from prior period. lin = raw levels, no transformation.
- **frequency**: Aggregation level. m = monthly, q = quarterly, a = annual. If this is lower than the series' native frequency, FRED averages or sums automatically.
- **sort_order**: asc = oldest first, desc = newest first.

Step 3: Connect Your Keys via the Settings Panel

The dashboard has a built-in **API** button in the top-right corner of the top bar. Click it to open the settings panel. You'll see three fields:

- **FRED API Key** (primary) -- powers all CPI, PPI, and employment data. FRED does not support CORS, so locally you run server.py which proxies calls automatically. For GitHub Pages, you deploy a Cloudflare Worker (see the API

guide).

- **BEA API Key** (optional) -- adds GDP and PCE (Personal Consumption Expenditures) data. BEA is the only one of the three that supports CORS, so it works directly in the browser with no proxy.
- **BLS API Key** (optional) -- stored for use with a proxy server. BLS does *not* support CORS either. Your key is saved in `localStorage` for use with the same proxy that handles FRED. FRED already mirrors all BLS data with a ~30 minute delay, so this is only needed for same-morning releases.

Paste your FRED key, click **Save & Connect**, and the dashboard immediately fires 16 parallel API calls, replaces the static data, and switches to live mode. The status badge in the top bar changes from "STATIC DATA" to "FRED LIVE" (or "FRED + BEA LIVE" if you also entered a BEA key). Data auto-refreshes every 5 minutes.

KEY SECURITY

Your keys are stored in **localStorage**, which is browser-only and domain-specific. They never appear in your source code, Git history, or GitHub repository. Anyone visiting your GitHub Pages URL sees static data until they enter their own keys. Even inspecting the page source reveals nothing -- the keys exist only in the browser's storage, not in the HTML/JS. This is the standard pattern for client-side API key management.

You can also click **Test All Keys** to verify each connection before saving, or **Disconnect All** to wipe stored keys and revert to static data.

Step 4: How the Fetch Function Works (Already Built In)

You don't need to write this code -- it's already in the dashboard. But understanding how it works is essential for modifying or extending the terminal. Here is the core function that calls FRED:

```
async function fetchFRED(seriesId, units = 'pc1', frequency = 'q') {
  const url = `${FRED_BASE}`
    + `?series_id=${seriesId}`
    + `&api_key=${FRED_API_KEY}`
    + `&file_type=json`
    + `&observation_start=2020-01-01`
    + `&units=${units}`
    + `&frequency=${frequency}`
    + `&sort_order=asc`;

  try {
    const response = await fetch(url);
    const json = await response.json();

    if (json.error_code) {
      console.error(`FRED error: ${json.error_message}`);
      return [];
    }

    return json.observations
      .filter(o => o.value !== '.')
      .map(o => ({
        date: o.date,
        value: parseFloat(o.value)
      }));
  } catch (err) {
    console.error(`Network error for ${seriesId}:`, err);
    return [];
  }
}
```

The function accepts defaults for units and frequency, so `fetchFRED('CPIAUCSL')` calls it with quarterly YoY percent changes — which is what 80% of the dashboard needs. For different needs, override as `fetchFRED('PAYEMS', 'chg', 'm')` to get monthly level changes.

The `filter(o => o.value !== '.')` line deserves explanation. FRED uses a period character (.) to represent missing observations — for example, when a release was cancelled or delayed. If we tried to `parseFloat('.')`, we'd get NaN (Not a Number), which would break our charts. Filtering out these missing values prevents that.

Step 5: How `loadAllData()` Works (Already Built In)

When you click **Save & Connect** in the settings panel, the dashboard calls `loadAllData()`. This function fires all API calls in parallel, reshapes the results into the format the dashboard expects, and triggers a re-render. Here is a simplified version of what runs under the hood:

```
async function loadAllData() {
  const [cpiQ, cpiA, ppiQ, unrate, nfp, ahe] = await Promise.all([
    fetchFRED('CPIAUCSL', 'pc1', 'q'),
    fetchFRED('CPIAUCSL', 'pc1', 'a'),
    fetchFRED('PPIFIS', 'pc1', 'q'),
    fetchFRED('UNRATE', 'lin', 'q'),
    fetchFRED('PAYEMS', 'chg', 'q'),
    fetchFRED('CES0500000003', 'pc1', 'q'),
  ]);

  DATA.cpi.quarterly = toQuarterlyMap(cpiQ);
  DATA.cpi.annual = toAnnualMap(cpiA);
  DATA.ppi.quarterly = toQuarterlyMap(ppiQ);
  DATA.unrate.quarterly = toQuarterlyMap(unrate);
  DATA.nfp.quarterly = toQuarterlyMap(nfp);
  DATA.ahe.quarterly = toQuarterlyMap(ahe);

  buildSummaryCards();
  buildTicker();
  renderView(currentView);

  console.log('[OK] Live data loaded at', new Date().toLocaleTimeString());
}

loadAllData();
setInterval(loadAllData, 5 * 60 * 1000); // Refresh every 5 minutes
```

The `Promise.all()` Pattern

`Promise.all()` takes an array of Promises and returns a new Promise that resolves when *all* of them resolve. This lets us fire six API calls simultaneously instead of sequentially. If each call takes 200ms, sequential execution would take 1.2 seconds — with `Promise.all()`, we're done in ~250ms (bottlenecked by the slowest call, not the sum). This is called **concurrency**.

CHAPTER 17

Setting Up Your Machine

Every tool you need, where to download it, and how to install it on Windows and Mac.

The terminal is a single HTML file that runs in any browser. You don't *need* to install anything to view it -- just double-click index.html. But to run a proper local development server, edit code with hot-reload, connect live APIs, and deploy to GitHub, you'll want four tools: **Python**, **Node.js**, **VS Code**, and **Git**. All four are free. Each takes about five minutes to install.

Tool A: Python

Python gives you the simplest possible local web server -- one command, zero config. It is the recommended way to view the dashboard on localhost.

Download: <https://www.python.org/downloads/>

Windows

- Go to the link above and click the large **Download Python 3.x.x** button.
- Run the .exe installer.
- **CRITICAL:** On the very first screen, check the box labeled **Add python.exe to PATH**. This checkbox is at the bottom of the window and is unchecked by default. If you miss it, the python command will not work in Command Prompt.
- Click **Install Now** (the default option is fine).
- Wait for the install to complete, then click **Close**.

Verify -- open Command Prompt (Win + R, type cmd, Enter):

```
python --version
```

WINDOWS ISSUE

If typing **python** opens the Microsoft Store instead of Python, go to Settings > Apps > App execution aliases and turn OFF the toggles next to **python.exe** and **python3.exe**. Close and reopen Command Prompt and try again.

Mac

Python 3 is pre-installed on macOS 12.3 (Monterey) and above. Open **Terminal** (press Cmd + Space, type Terminal, press Enter) and verify:

```
python3 --version
```

If you see a version number (e.g., Python 3.11.6), you're set. If you get "command not found" or it shows Python 2, install Python 3 using one of these methods:

- **Option 1 (easiest):** Download the macOS installer from python.org/downloads/macos. Run the .pkg file and follow the prompts.

- **Option 2 (Homebrew):** If you have Homebrew installed, run: `brew install python3`
- **Option 3:** Running `python3` on a fresh Mac may trigger the Xcode Command Line Tools installer automatically. Click **Install** when prompted -- this installs Python 3 along with Git.

**MAC
NOTE**

On Mac, always use **python3** (not **python**). The bare **python** command may point to an old system Python 2 or not exist at all. Similarly, use **pip3** instead of **pip**.

Tool B: Node.js

Node.js is a JavaScript runtime. Installing it gives you **npm** (Node Package Manager), the standard way to install JavaScript tools.

Download: <https://nodejs.org/en/download>

Windows

- Click the **LTS** (Long Term Support) version. Download the .msi installer.
- Run the installer -- accept all defaults, click Next through every screen.
- Click **Install**, then **Finish**.

Verify -- open a **new** Command Prompt:

```
node --version
npm --version
```

Mac

- **Option 1 (installer):** Download the .pkg from nodejs.org/en/download. Run it and follow the prompts. It installs both node and npm.
- **Option 2 (Homebrew):** `brew install node`

Verify -- open a **new** Terminal window:

```
node --version
npm --version
```

Tool C: Visual Studio Code

VS Code is a free code editor. With the **Live Server** extension, it lets you edit dashboard code and see changes instantly in your browser.

Download: <https://code.visualstudio.com/download>

Windows

- Click the **Windows** button. Run the installer.
- **Recommended:** Check these boxes during install: "Add Open with Code to file context menu", "Add Open with Code to directory context menu", and "Add to PATH".
- Click **Install**, then **Finish**.

Mac

- Click the **macOS** button. This downloads a .zip file.
- Open the zip -- it extracts an app called **Visual Studio Code.app**.
- **Drag it into your Applications folder.** This is important -- if you run it from Downloads, it won't update properly.
- Open VS Code from Applications (or Spotlight: Cmd + Space, type "Visual Studio Code").
- To add the code command to Terminal: open VS Code, press Cmd + Shift + P, type **Shell Command: Install 'code' command in PATH**, and press Enter.

Install Live Server (Both Platforms)

- Open VS Code.
- Press Ctrl + Shift + X (Windows) or Cmd + Shift + X (Mac) to open Extensions.
- Search **Live Server**. Install the one by **Ritwick Dey**.
- A **Go Live** button appears in the bottom-right status bar.

Tool D: Git

Git tracks changes to your files and lets you push code to GitHub for hosting.

Download: <https://git-scm.com/downloads>

Windows

- Click **Windows**. The download starts automatically.
- Run the installer. **Accept all defaults** -- they are safe and correct.
- Optionally switch the default editor to **VS Code** on the editor selection screen.
- Click **Install**, then **Finish**.

After install, you'll have **Git Bash** (a Linux-style terminal) in your Start menu.

Mac

Git is usually already installed with Xcode Command Line Tools. Open Terminal and type:

```
git --version
```

If Git is installed, you'll see a version number. If not, macOS will pop up a dialog asking "The 'git' command requires the command line developer tools. Would you like to install them?" Click **Install** and wait 2-3 minutes. This installs Git, Python 3, and a C compiler in one step.

Alternatively, install via Homebrew: `brew install git`.

Configure Git (Both Platforms)

Run these in Terminal (Mac) or Git Bash / Command Prompt (Windows):

```
git config --global user.name "Your Name"
git config --global user.email "kris@ghoststrategies.io"
```

Quick Reference: All Download Links

CHAPTER 18

Running Locally

Four ways to view the dashboard on Windows and Mac.

Option 1: Double-Click (Zero Install)

The dashboard is a single self-contained HTML file. Double-click index.html and it opens in your default browser. No server, no install, nothing.

Windows: start index.html in Command Prompt.

Mac: open index.html in Terminal.

Option 2: Python Server (Recommended)

Requires: Python installed (see Chapter 17, Tool A)

Windows (Command Prompt)

```
:: Navigate to the project folder
cd %USERPROFILE%\Desktop\economic-terminal

:: Start the server
python -m http.server 8080
```

Mac (Terminal)

```
# Navigate to the project folder
cd ~/Desktop/economic-terminal

# Start the server (note: python3, not python)
python3 -m http.server 8080
```

On both platforms, open **http://localhost:8080** in your browser. Press Ctrl + C (Windows) or Control + C (Mac) to stop the server.

Problem	Platform	Solution
'python' is not recognized	Windows	Reinstall Python, check 'Add to PATH'.
python opens Microsoft Store	Windows	Settings > Apps > App execution aliases > turn OFF python.exe.
'python3' command not found	Mac	Run: xcode-select --install to install Command Line Tools.
Port 8080 already in use	Both	Use a different port: python3 -m http.server 9000

Option 3: VS Code + Live Server (Best for Editing)

Requires: VS Code + Live Server extension (see Chapter 17, Tool C)

Windows

- Right-click the economic-terminal folder in File Explorer and select **Open with Code**.
- Click index.html in the left panel, then click **Go Live** in the bottom-right.
- Browser opens at <http://127.0.0.1:5500/index.html>. Save edits with Ctrl + S to auto-refresh.

Mac

- Open Terminal and type code ~/Desktop/economic-terminal to open the folder in VS Code. (Requires the shell command installed -- see Chapter 17.)
- Alternatively, open VS Code, go to **File > Open Folder**, and navigate to the project.
- Click index.html in the left panel, then click **Go Live**.
- Save edits with Cmd + S to auto-refresh the browser.

Option 4: Node.js Server

Requires: Node.js installed (see Chapter 17, Tool B)

Windows (Command Prompt)

```
npm install -g http-server
cd %USERPROFILE%\Desktop\economic-terminal
http-server -p 8080
```

Mac (Terminal)

```
npm install -g http-server
cd ~/Desktop/economic-terminal
http-server -p 8080
```

On both platforms, open <http://localhost:8080>.

CHAPTER 19

Deploying to GitHub Pages

From local folder to live public URL in 10 minutes.

Requires: Git installed (see Chapter 17, Tool D) and a GitHub account.

Step 1: Create a GitHub Account

Go to <https://github.com> and click **Sign Up**. Enter a username, email, and password, then verify your email. This step is identical on both platforms.

Step 2: Create a New Repository

- Go to <https://github.com/new>.
- Repository name: economic-terminal.
- Set it to **Public** (required for free GitHub Pages).
- Do **not** check "Add a README" (we already have one).
- Click **Create repository**. Leave this tab open.

Step 3: Push Your Code

The Git commands are the same on both platforms -- the only difference is the terminal application and the path syntax.

Windows (Git Bash -- recommended)

Open **Git Bash** from the Start menu and run:

```
cd ~/Desktop/economic-terminal
git init
git add .
git commit -m "Initial commit: Economic Indicators Terminal v2.1"
git branch -M main
git remote add origin https://github.com/YOUR_USERNAME/economic-terminal.git
git push -u origin main
```

Windows (Command Prompt -- alternative)

If you prefer Command Prompt over Git Bash, the only change is the path syntax:

```
cd %USERPROFILE%\Desktop\economic-terminal
git init
git add .
git commit -m "Initial commit: Economic Indicators Terminal v2.1"
git branch -M main
git remote add origin https://github.com/YOUR_USERNAME/economic-terminal.git
git push -u origin main
```

Mac (Terminal)

Open **Terminal** (Cmd + Space, type Terminal) and run:

```
cd ~/Desktop/economic-terminal
git init
git add .
git commit -m "Initial commit: Economic Indicators Terminal v2.1"
git branch -M main
git remote add origin https://github.com/YOUR_USERNAME/economic-terminal.git
git push -u origin main
```

The Mac commands are identical to Git Bash on Windows because both use Unix-style paths. Replace YOUR_USERNAME with your actual GitHub username in all three examples.

Step 4: Authentication (Personal Access Token)

GitHub no longer accepts plain passwords. When Git prompts for a password, you need a **Personal Access Token**. This applies to both Windows and Mac:

- Go to <https://github.com/settings/tokens>.
- Click **Generate new token (classic)**.
- Name it terminal-deploy, check the **repo** scope.
- Click **Generate token** and **copy it immediately** (you won't see it again).
- When Git asks for your password, **paste the token** instead.

MAC KEYCHAIN

On Mac, after you enter the token once, macOS Keychain stores it automatically. You won't be prompted again for future pushes. On Windows, Git Credential Manager (installed with Git) does the same thing. If you ever need to update a stored token, search for "Keychain Access" on Mac or "Credential Manager" on Windows.

Step 5: Enable GitHub Pages

This step is done entirely in the browser, so it's identical on both platforms:

- Go to your repository: https://github.com/YOUR_USERNAME/economic-terminal.
- Click **Settings** (gear icon, top right of the repo page).
- In the left sidebar, click **Pages**.
- Under Source, select **Branch: main** and **Folder: / (root)**.
- Click **Save**. Wait 1-2 minutes.
- Your live URL appears: https://YOUR_USERNAME.github.io/economic-terminal/

Pushing Updates (Both Platforms)

After editing any file, push changes with three commands:

```
git add .
git commit -m "Update: description of what you changed"
git push
```

These commands are identical on Windows (Git Bash) and Mac (Terminal). GitHub Pages automatically redeploys within 1-2 minutes.

CHAPTER APPENDIX A

Series ID Dictionary

Every FRED series used by the terminal, plus high-value additions.

Inflation Series

Series ID	Description	Frequency
CPIAUCSL	CPI All Items, SA	Monthly
CPILFESL	CPI Core (ex F&E), SA	Monthly
CPIUFDSL	CPI Food	Monthly
CPIENGSL	CPI Energy	Monthly
CUSR0000SAH1	CPI Shelter	Monthly
CUSR0000SEHA	CPI Rent of Primary Residence	Monthly
CUSR0000SEHC	CPI Owners Equivalent Rent	Monthly
CPIMEDSL	CPI Medical Care	Monthly
CUSR0000SAT	CPI Transportation	Monthly
PCEPI	PCE Price Index (Fed preferred)	Monthly
PCEPILFE	Core PCE	Monthly
PPIFIS	PPI Final Demand	Monthly
PPIFES	PPI Core (ex F&E&T)	Monthly
PPIFGS	PPI Final Demand Goods	Monthly
PPIFSS	PPI Final Demand Services	Monthly

Employment Series

Series ID	Description	Frequency
PAYEMS	Total Nonfarm Payrolls, SA	Monthly
USPRIV	Private NFP	Monthly
UNRATE	Unemployment Rate (U-3)	Monthly
U6RATE	U-6 Rate	Monthly
CIVPART	Labor Force Participation Rate	Monthly

CHAPTER APPENDIX B

CSS Property Cheat Sheet

The properties most used in the terminal, with definitions.

Property	What It Controls	Example
background	Fill color or image	background: #111820;
color	Text color	color: #ff9500;
font-family	Typeface stack	font-family: 'JetBrains Mono';
font-size	Text size	font-size: 14px;
font-weight	Text thickness	font-weight: 600;
padding	Inner space	padding: 16px;
margin	Outer space	margin: 10px 0;
border	Edge line	border: 1px solid #1e2a36;
border-radius	Rounded corners	border-radius: 4px;
display	Layout mode	display: grid;
grid-template-columns	Grid column sizes	grid-template-columns: 1fr 1fr;
gap	Grid/flex spacing	gap: 12px;
position	Positioning mode	position: sticky;
top, right, bottom, left	Offset positions	top: 0;
z-index	Stacking order	z-index: 100;
overflow	Content clipping	overflow: hidden;
text-align	Text alignment	text-align: right;
text-transform	Upper/lowercase	text-transform: uppercase;
letter-spacing	Character spacing	letter-spacing: 1.5px;
line-height	Vertical text spacing	line-height: 1.4;
opacity	Transparency 0–1	opacity: 0.8;
transform	Scale/rotate/translate	transform: translateX(-50%);
transition	Animate property change	transition: all 0.2s;
animation	Apply keyframe animation	animation: pulse 2s infinite;
cursor	Pointer style on hover	cursor: pointer;

CHAPTER APPENDIX C

JavaScript Concepts Glossary

Every JS term used in this guide, defined.

Array — An ordered list of values. Accessed by numeric index. Example: `["a", "b", "c"][0]` returns "a".

Arrow function — A compact function syntax: `(x) => x * 2`. Unlike regular functions, arrow functions inherit "this" from their enclosing scope.

async/await — Syntax for writing asynchronous code in a linear style. "async" marks a function; "await" pauses execution until a Promise resolves.

Callback — A function passed to another function to be called later. Common in event handlers and older async APIs.

Closure — A function that remembers variables from the scope in which it was defined, even after that scope has exited.

const — Declares a variable that cannot be reassigned. Still allows mutation of objects and arrays assigned to it.

DOM — Document Object Model. The browser's in-memory tree representation of your HTML.

fetch() — Built-in browser function for making HTTP requests. Returns a Promise.

JSON — JavaScript Object Notation. A lightweight text format for structured data. Standard API data format.

let — Declares a variable that can be reassigned. Block-scoped, unlike "var".

map() — Array method that applies a function to every element and returns a new array with the results.

Object — A collection of key-value pairs. Keys are strings; values can be any type.

parseFloat() — Converts a string to a floating-point number. "3.14" becomes 3.14.

Promise — A value that will be available in the future. Returned by `fetch()` and other async operations.

Promise.all() — Takes an array of Promises; returns a single Promise that resolves when all of them do. Enables concurrent execution.

Template literal — A string wrapped in backticks that supports multi-line content and `${expression}` interpolation.

try/catch — Syntax for handling errors. Code in the "try" block runs; if it throws, the "catch" block executes with the error.

var — Older variable declaration. Function-scoped rather than block-scoped. Avoid in new code.

CHAPTER APPENDIX D

Troubleshooting Common Errors

What to do when things go wrong.

Q: The dashboard shows blank charts after I added API calls

A: Most likely cause: your API key is wrong or missing. Open the browser console (F12 → Console tab) and look for red error messages. You'll usually see "FRED error: API key not valid." Double-check that the key you pasted is exactly 32 characters with no trailing spaces or quotes.

Q: I get a CORS error in the console

A: Neither FRED nor BLS return CORS headers, so browsers block direct calls. Run `server.py` locally (it proxies calls automatically), or deploy a Cloudflare Worker for GitHub Pages. BEA is the only API that works directly in the browser. See [api-guide.html](#) for the Worker template.

Q: Chart.js says "Chart is not defined"

A: The Chart.js library didn't load. Check your internet connection, and verify the `<script src=...">` line in your HTML points to a valid CDN URL. If cdnjs is down, you can use the jsdelivr mirror instead.

Q: My GitHub Pages site shows 404

A: GitHub Pages needs a few minutes after you enable it. Also, the main page must be named exactly "index.html" (case-sensitive on some systems). Verify the branch is set to "main" and the folder is "/" (root) in Settings → Pages.

Q: localStorage.setItem doesn't persist

A: localStorage is domain-specific. If you're opening the file directly with a `file://` URL instead of through a local server, localStorage may not work consistently. Use a local server (`python -m http.server 8080`) for proper testing.

Q: FRED returns fewer observations than I expected

A: Check your "frequency" parameter. Requesting `frequency=q` (quarterly) from a monthly series returns fewer rows because FRED aggregates. This is usually what you want — but if you need monthly granularity, use `frequency=m`.

Q: Percentage values look wrong (e.g., 0.028 instead of 2.8)

A: You're receiving raw decimals instead of percents. Add `units=pc1` or `units=pch` to your FRED URL. These tell FRED to return the series pre-transformed to percent change.

Q: The ticker bar isn't scrolling

A: The scroll animation is CSS-driven with `@keyframes`. If it's frozen, either the CSS didn't load or a JavaScript error is preventing the page from rendering fully. Check the console for errors.

Q: Dates are off by one day

A: JavaScript's Date object interprets ISO strings like "2024-01-01" as UTC midnight, which may display as Dec 31 in your local timezone. For economic data where only the month matters, use string parsing instead of Date objects, or extract year/month directly from the string with `substring()`.

Q: Rate limit errors (HTTP 429)

A: You're hitting FRED too fast. The limit is 120 requests/minute. If you're polling aggressively, add a `setTimeout` delay between calls. The terminal's 5-minute refresh interval is well under this limit.

END OF GUIDE

You Now Know How It Works

If you've read through to this page, you understand the terminal not as a mysterious black box but as a carefully composed system of three text files' worth of HTML, CSS, and JavaScript talking to a public government API.

The patterns you've learned here — semantic data modeling, CSS custom properties, asynchronous data fetching, separation of concerns between structure / style / behavior — are not specific to economic dashboards. They are the fundamentals of modern web development. A React application, a Vue dashboard, a Svelte SPA — all of them use these same building blocks at their core. The frameworks just wrap them in different syntax.

Your NVIDIA valuation workbench, your StockPulse dashboard, and any future Ghost Strategies tool you build will use variations of these exact techniques. The skill here isn't memorizing the terminal's code — it's developing the muscle to read *any* frontend code and understand what it's doing.

NEXT STEPS

(1) Click the **API** button in the dashboard and enter your FRED key -- live data flows instantly with no code changes. (2) Register for a BEA key at apps.bea.gov/api/signup and add it in the settings panel for GDP data. (3) Deploy to GitHub Pages following Chapter 19. (4) Consider porting the terminal to React with Vite, which would give you component reusability, hot module reload, and easier state management. The underlying concepts stay identical; only the syntax changes.

GHOST STRATEGIES LLC

Economic Indicators Terminal — Instructional Guide — April 2026